



Collective communication pattern

Prof. Luigi Asprino

1



Introduzione ai Collective Communication Pattern

Definizione e ruolo

- **Collective communication pattern:** tutti i processi di un gruppo partecipano a un'unica operazione globale
- Differisce dalle comunicazioni **punto-punto**: nessun mittente/destinatario esplicito
- Operazione **atomica**: la correttezza richiede la partecipazione coordinata di ogni processo
- Nati nell'**HPC**; riadattati per il training distribuito iterativo

Perché servono le comunicazioni collettive

- **Data-parallel training:** ogni processo calcola gradienti sulla propria partizione
- Gradienti devono essere **aggregati** correttamente a ogni iterazione
- Comunicazione inefficiente $\Rightarrow$ **collo di bottiglia**
- Forniscono **barriera di sincronizzazione** implicita
- Ottimizzano **banda** e topologia di rete automaticamente

Modello formale

Insieme di N processi: $P = \{p_0, \dots, p_{N-1}\}$. Funzione collettiva globale:

$$C : (x_0, \dots, x_{N-1}) \longrightarrow (y_0, \dots, y_{N-1})$$

- **Sincrona**: ogni processo attende il completamento
- Stessa $C \Rightarrow$ molteplici implementazioni con diversi trade-off
- **Communicator**: definisce il gruppo di processi partecipanti

Primitive fondamentali di comunicazione collettiva: Broadcast e Reduce

Broadcast – radice p_r distribuisce a tutti:

$$y_i = x_r \quad \forall i$$

Replica **parametri iniziali** e iperparametri.

Reduce – aggrega e consegna il risultato a un solo processo:

$$y_r = \bigoplus_{i=0}^{N-1} x_i$$

Richiede $\oplus$ **associativa**; usata per metriche di validazione.

Primitive fondamentali di comunicazione collettiva: All-Reduce, Gather, All-Gather, Scatter

All-Reduce – risultato disponibile a tutti:

$$y_i = \bigoplus_{j=0}^{N-1} x_j \quad \forall i$$

Primitiva centrale per aggregare i **gradienti** nel training data-parallel.

Gather: $y_r = (x_0, \dots, x_{N-1})$ su radice

All-Gather: $y_i = (x_0, \dots, x_{N-1}) \forall i$

Scatter: $y_i = x_i$ — il radice distribuisce una **partizione** a ciascun processo

Algoritmi di implementazione delle operazioni collettive

Tree-based, Ring-based, Recursive Doubling

Tree-based: albero binario/k-ario; **latenza** $\mathcal{O}(\log N)$; ottimale per messaggi piccoli; squilibrio nei nodi interni.

Ring-based: anello; **bandwidth-optimal**; latenza $\mathcal{O}(N)$; adatto a gradienti grandi.

Recursive Doubling: convergenza in $\log N$ fasi; bassa **latenza** per messaggi piccoli.

Scelta adattiva: il **runtime** (NCCL, MPI) seleziona dinamicamente la strategia.

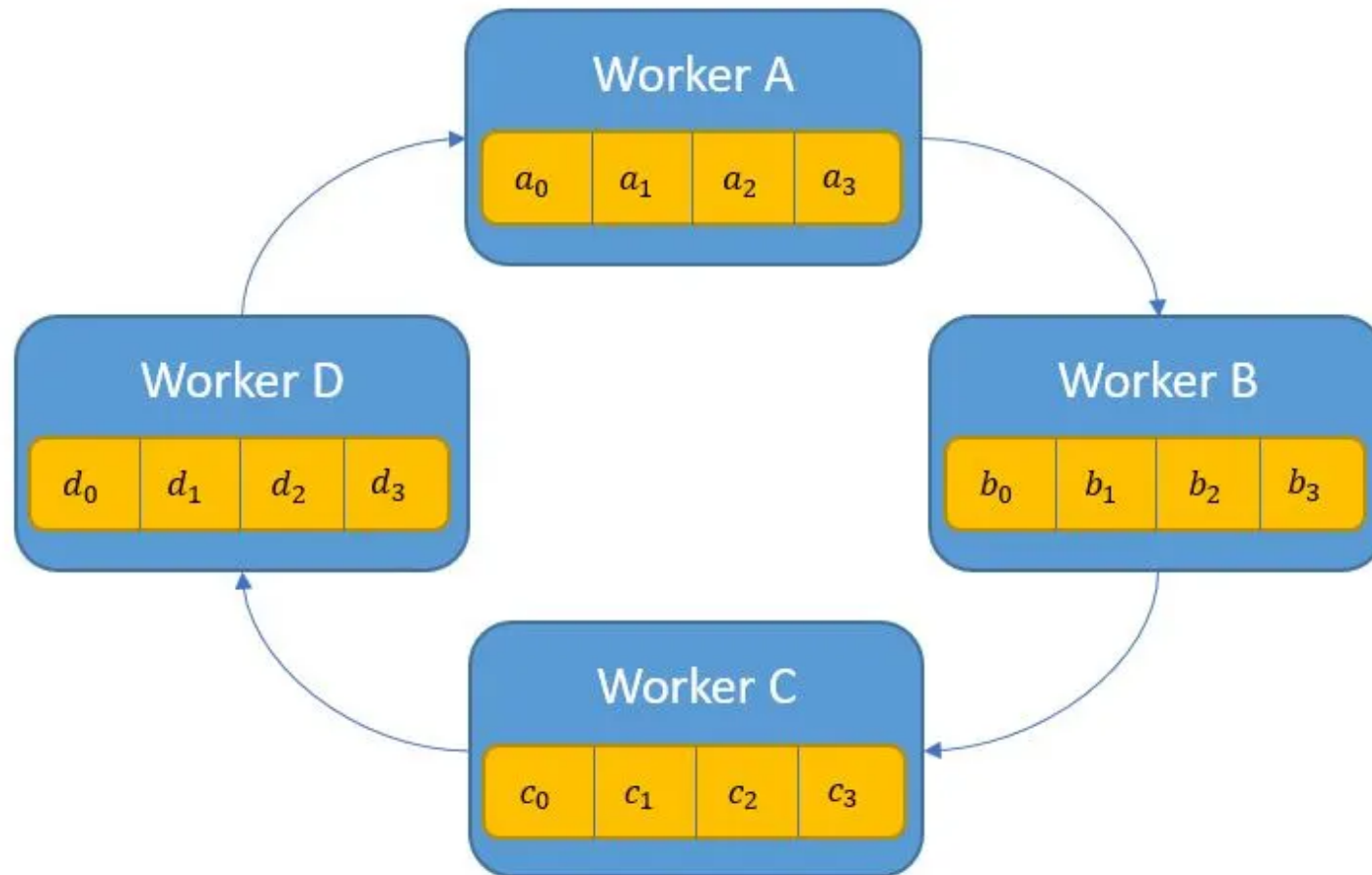
Ring-Based All-Reduce nel training multi-GPU

N GPU, vettore $g^{(i)}$ suddiviso in N blocchi; due fasi:

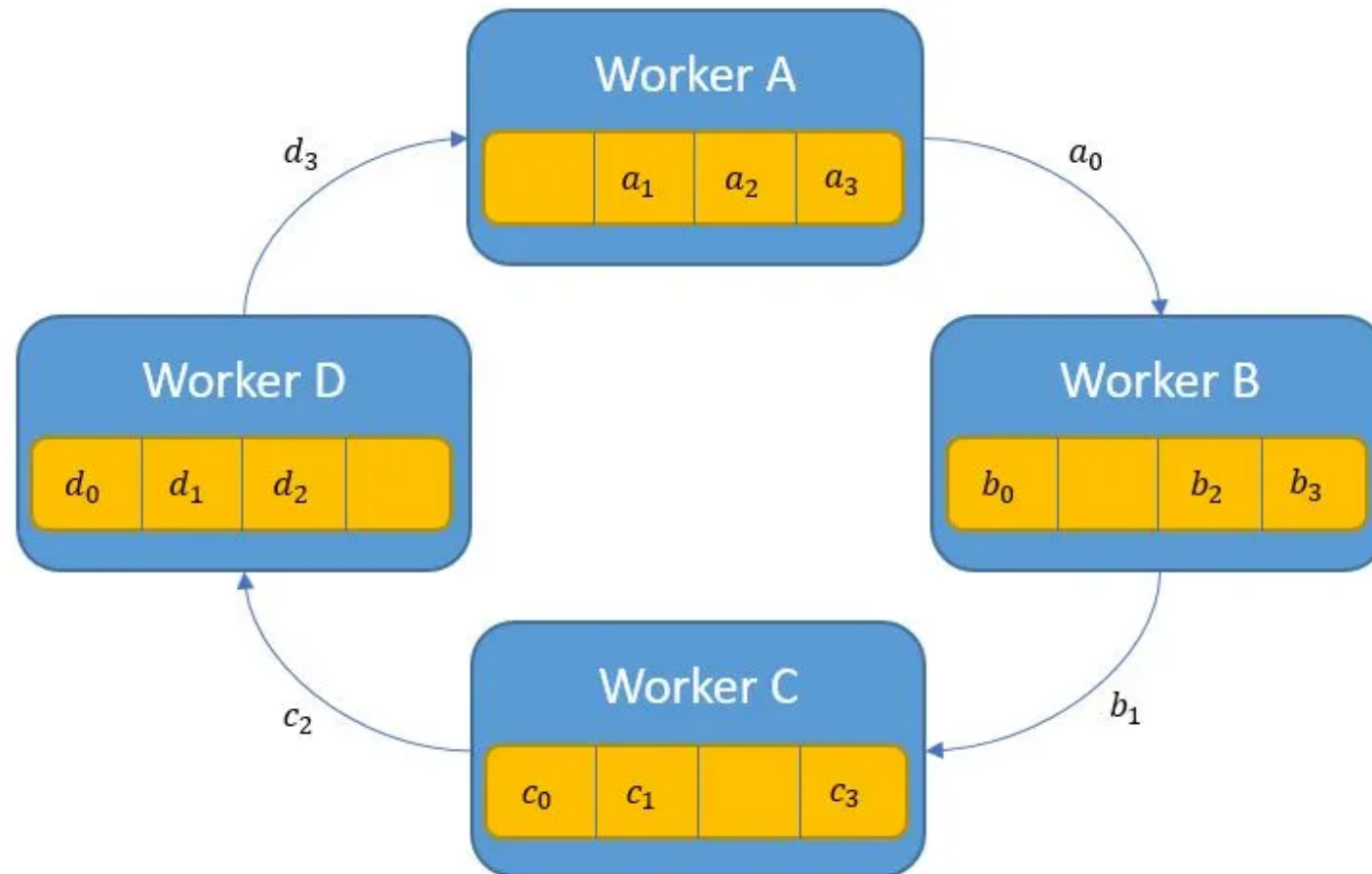
1. **Reduce-Scatter** ($N - 1$ passi): ogni processo invia un blocco al successore e aggrega quello ricevuto.
2. **All-Gather** ($N - 1$ passi): i blocchi circolano fino a ricostruire $g = [g_0, \dots, g_{N-1}]$.

Volume per processo: $2 \cdot \frac{N-1}{N} \cdot |x| \Rightarrow$ **bandwidth-optimal**.

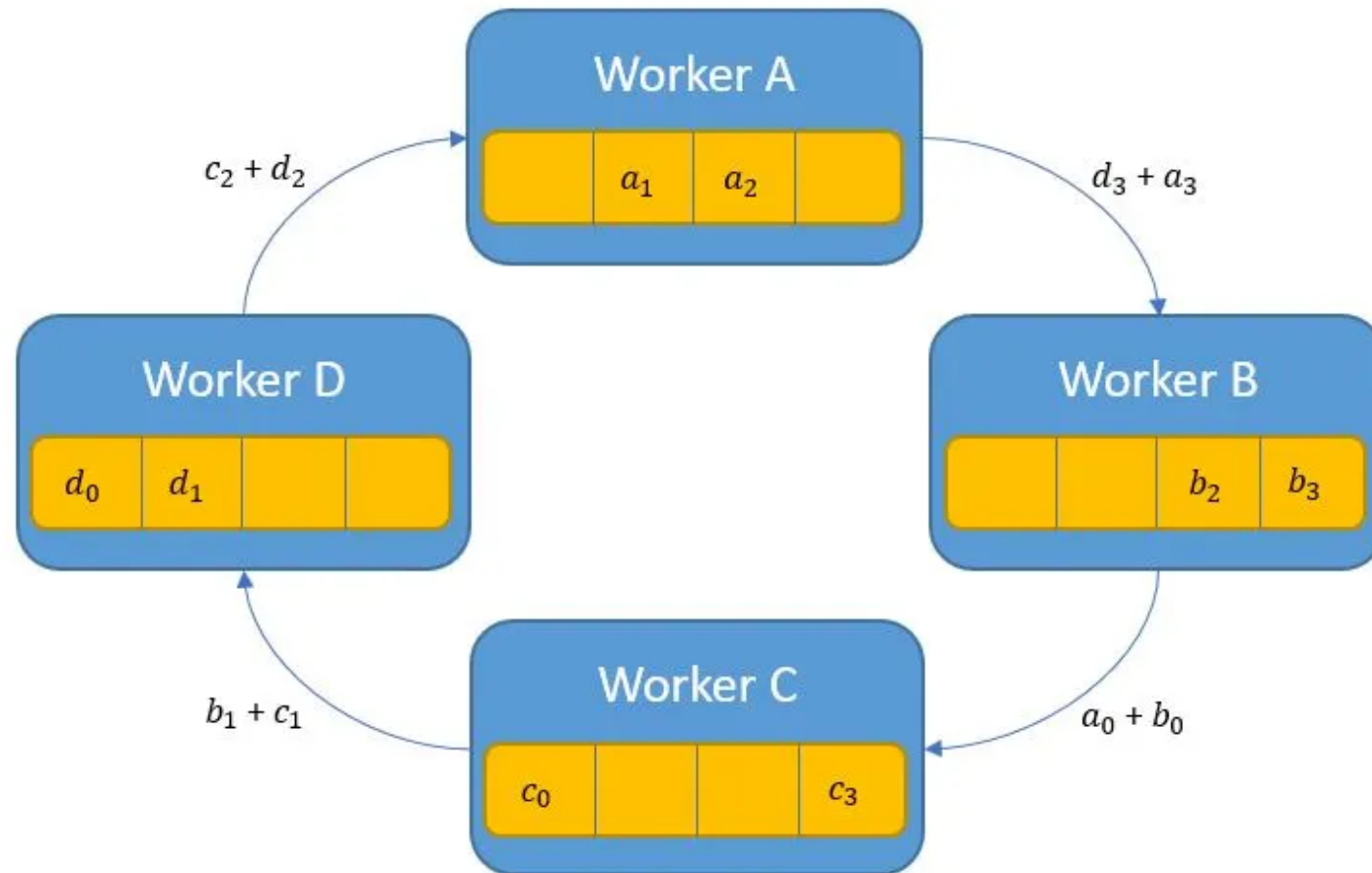
Ring-Based All-Reduce: visualizzazione



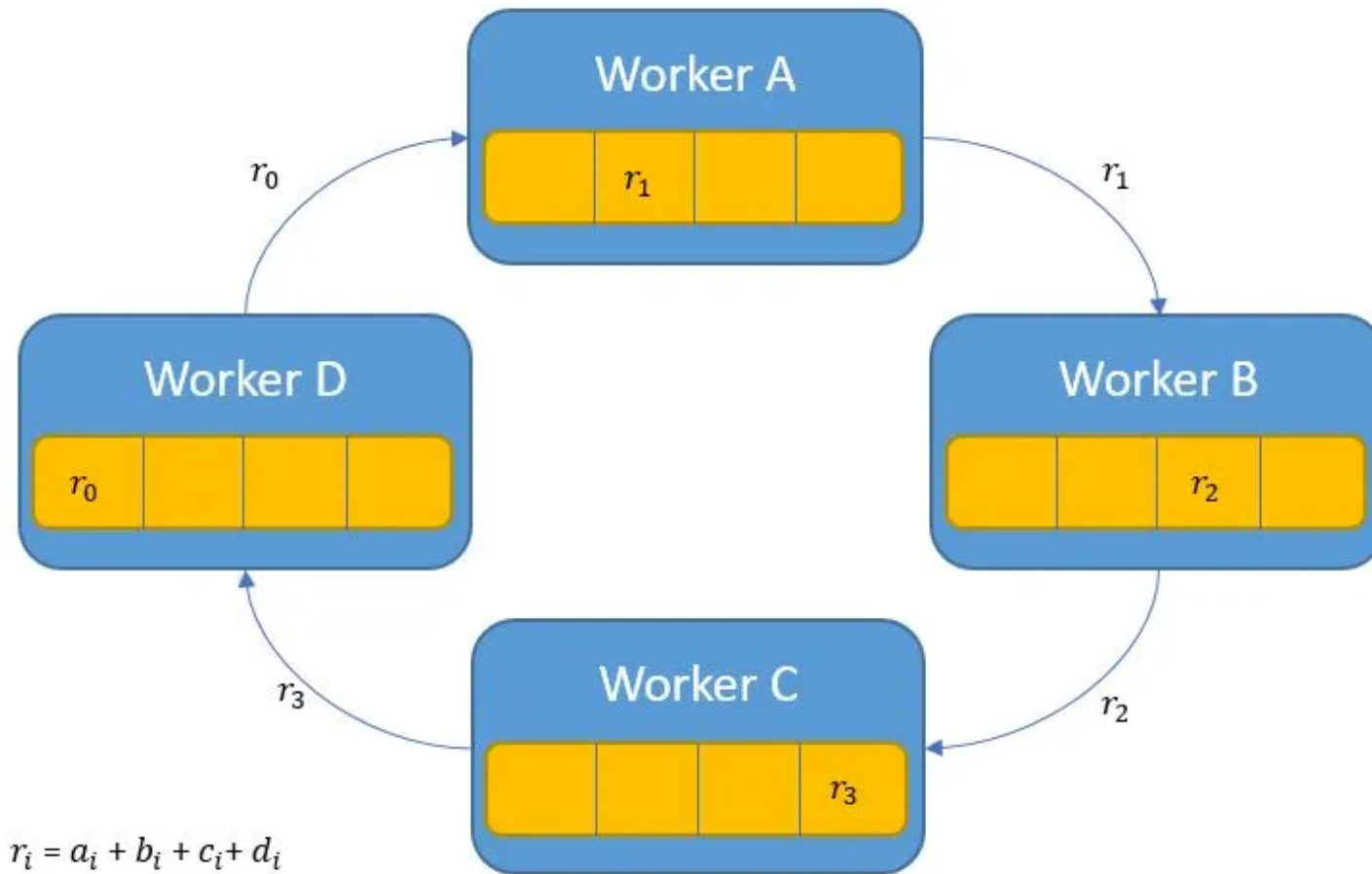
Ring-Based All-Reduce: visualizzazione



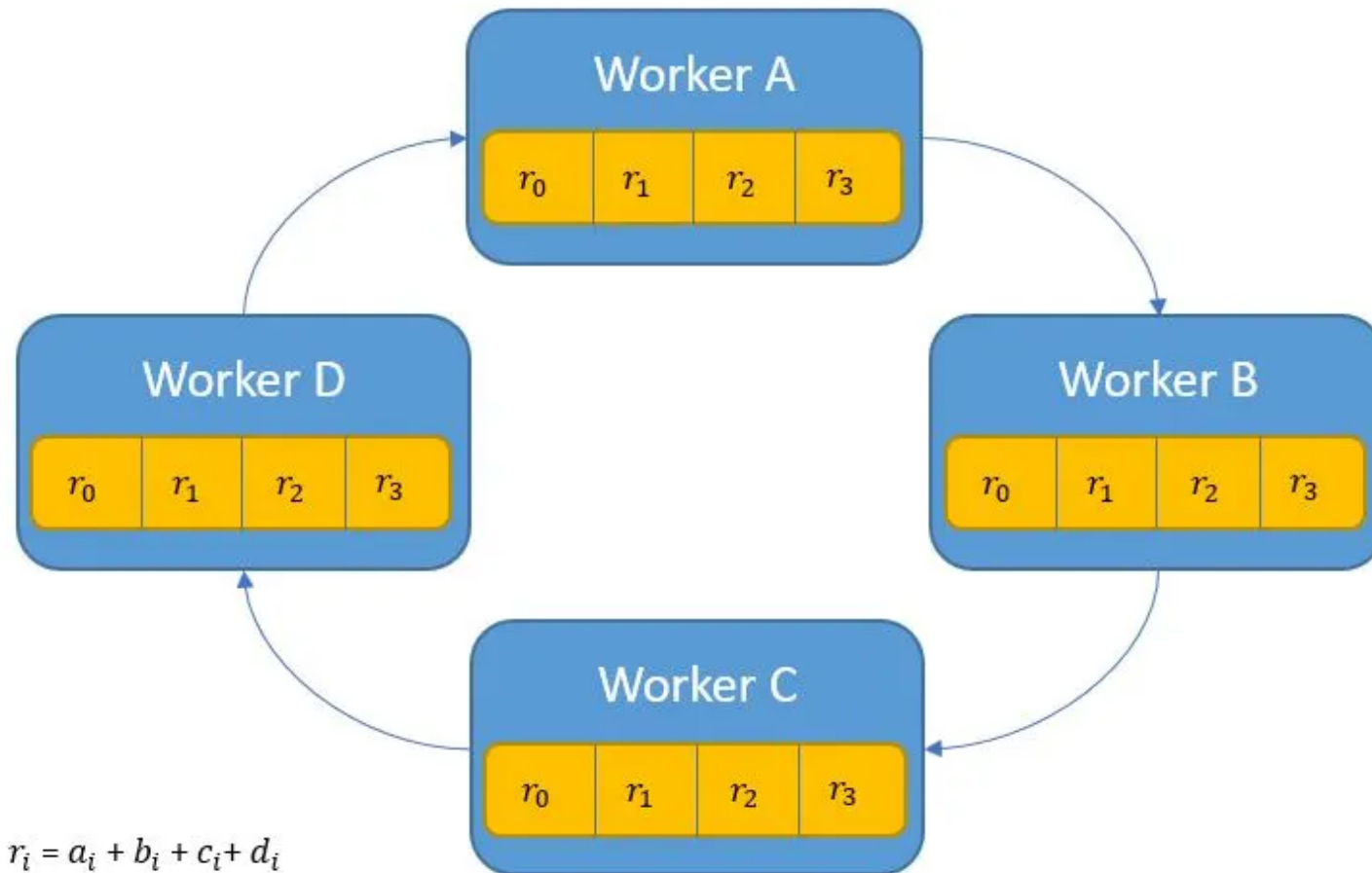
Ring-Based All-Reduce: visualizzazione



Ring-Based All-Reduce: visualizzazione



Ring-Based All-Reduce: visualizzazione



1

Impatto sulle prestazioni del training distribuito

Modello di costo e scalabilità

Costo della all-reduce (**latenza** α , costo per byte β):

$$T_{\text{all-reduce}} \approx \alpha \cdot f(N) + \beta \cdot g(N) \cdot |x|$$

Tempo per iterazione: $T_{\text{iter}} = T_{\text{comp}} + T_{\text{comm}}$

- Aumentando N : $T_{\text{comp}} \downarrow$ ma T_{comm} può crescere $\Rightarrow$ **scalabilità** si satura
- **Overlap asincrono** comunicazione/computazione per nascondere il costo

Confronto con il Parameter Server Pattern

Collective vs Parameter Server

Aspetto	Collective	Parameter Server
Topologia	decentralizzata	centralizzata
Sincronia	sincrono	anche asincrono
Collo di bottiglia	assente	nodo PS saturo
Scalabilità	ottima su HPC	flessibile
Staleness	assente	rischio

I collective pattern eliminano il **nodo centrale** ma richiedono training **sincrono**.

Considerazioni su fault tolerance ed elasticità

Fault Tolerance ed Elasticità

Fault tolerance: un nodo fallito blocca l'intera operazione.

- **Checkpointing:** stato del modello salvato periodicamente
- Riorganizzazione del **communicator**: esclude i nodi falliti

Elasticità: aggiunta/rimozione di nodi a runtime è complessa.

- Fasi di **riconfigurazione esplicita** con ridistribuzione parametri
- Il **Parameter Server** è più flessibile in ambienti eterogenei

Concetti principali della lezione

- **Collective pattern**: operazione atomica globale su un gruppo di processi
- **Primitive**: Broadcast, Reduce, All-Reduce, Gather, Scatter
- $\oplus$ **associativa**; **sincronizzazione** implicita tra processi
- **Ring-based All-Reduce**: Reduce-Scatter + All-Gather; bandwidth-optimal
- $T_{\text{iter}} = T_{\text{comp}} + T_{\text{comm}}$: comunicazione limita la **scalabilità**
- No **collo di bottiglia** vs PS; limite: **elasticità** e fault tolerance